

Randomiserede algoritmer til dataanalyse

Implementeringsprojekt

Hold 1 Gruppe 1

Lars V. Thorup	Victor Vangkilde Jørgensen
Department of Computer Science	Department of Computer Science
Universitetsparken 1	Universitetsparken 1
<code>rlp419@alumni.ku.dk</code>	<code>kft410@alumni.ku.dk</code>

Vitus N. Legarth
Department of Computer Science
Universitetsparken 1
`gfn320@alumni.ku.dk`

13. juni 2026

Del 1

Opgave 1

I denne opgave implementerede vi to basale universelle hashfunktioner.

Multiply-Shift

Ved Multiply-Shift beregnes hashværdien ved at multiplicere nøglen med en tilfældig ulige konstant, hvorefter man beholder de 64 mindst betydende bits. Dette sker ved at bitshiftes til højre.

```
1 public static ulong MultiplyShift(ulong x, ulong a, int l)
2 {
3     unchecked
4     {
5         ulong res = a * x;
6         return res >> (64 - l);
7     }
8 }
```

Multiply-Mod-Prime

For Multiply-Mod-Prime beregnes funktionen ud fra formelen

$$((ax + b) \bmod p) \bmod (2^l) \quad (1)$$

hvor p er et primtal, og a og b er heltal skarpt mindre end p og l er et positivt heltal mindre end 64. I vores tilfælde anvendes $p = 2^{89} - 1$.

```
1 public static ulong MultiplyModPrime(ulong x, BigInteger a,
2     BigInteger b, int l)
3 {
4     BigInteger p = (BigInteger.One << 89) - 1;
5     BigInteger r = (a * x + b);
6     BigInteger z = (r >> 89) + (r & p);
7     if (z >= p) z -= p;
8     z = z & ((BigInteger.One << l) - 1);
9     return (ulong)z;
10 }
```

Opgave 2

I denne opgave har vi konstrueret en hashtabel ved hjælp af chaining, hvilket betyder at der i hver spand er mange elementer, og at vi ikke bruger two-level hashing til at finde den ønskede nøgle, men at vi itererer over alle elementerne i spanden. Når hastabellen bygges, sættes hver ny nøgle forrest i listen.

Get()

Get-funktionen er operationen hvor vi leder igennem vores spand indtil vi finder den ønskede nøgle. Den er derfor implementeret som en simpel for-løkke og et if-statement.

```
1 public long Get(ulong x)
2 {
3     foreach (var entry in _table[_hash(x)])
4         if (entry.key == x) return entry.value;
5     return 0;
6 }
```

Set()

Set-funktionen tager en x-værdi og en v-værdi og leder igennem spanden efter x. Finder den x, sætter den værdien lig med den nye værdi v - finder den ikke x, tilføjer den (x,v) til spanden forrest i listen.

```
1 public void Set(ulong x, long v)
2 {
3     var list = _table[_hash(x)];
4     for (var node = list.First; node != null; node = node.Next)
5     {
6         if (node.Value.key == x) { node.Value = (x, v); return; }
7     }
8     list.AddFirst((x, v));
9 }
```

Increment()

Increment-funktionen tager en x-værdi og en d-værdi og leder en spand igennem for x-værdien. Finder den x, lægger den d oveni den eksisterende værdi - finder den ikke x, tilføjer den (x,v) til spanden forrest i listen.

```
1 public void Increment(ulong x, long d)
2 {
3     var list = _table[_hash(x)];
4     for (var node = list.First; node != null; node = node.Next)
5     {
6         if (node.Value.key == x) { node.Value = (x, node.Value.
          value + d); return; }
7     }
8     list.AddFirst((x, d));
9 }
```

Opgave 3

Vi vil nu sammenligne Multiply-Shift og Mulipty-Mod-Prime algoritmernes køretid. Vi anvender chaining-hashtabellen fra opgave 2 og laver en ny funktion ComputeSumOfSquares til at beregne kvadratsummen, angivet som S. Dette gøres ved at summere kvadraterne af frekvensen for alle unikke nøgler i tabellen.

```

1 public static long ComputeSumOfSquares(IEnumerable<Tuple<ulong,
   int>> stream, Func<ulong, ulong> hash, int l)
2 {
3     var table = new HashTableChaining(hash, l);
4     foreach (var item in stream)
5         table.Increment(item.Item1, item.Item2);
6
7     long sum = 0;
8     foreach (var v in table.AllValues())
9         sum += v * v;
10    return sum;
11 }

```

Tabellen nedenfor præsenterer resultaterne for beregningen med de to forskellige hash-funktioner over stigende værdier af l .

l	Multiply-Shift (ms)	Mod-Prime (ms)	Kvadratsum S
1	2584	8799	62549987368052
2	2496	9164	31274993684028
4	2460	9539	7818748421020
8	2513	9701	488671776540
16	3081	13476	1908932380
20	10387	18464	120236700
21	Timeout	Timeout	N/A

Kørslen viser at Multiply-Shift er hurtigere end Multiply-Mod-Prime for alle l , dog at forskellen formindskes når l bliver større. Multiply-Mod-Prime er afhængig af langsommere heltals- og modulo-operationer, hvor Multiply-Shift udelukkende kræver en basal bitoperation.

Matematisk og arkitekturmæssigt forklares konvergensens af funktionernes køretid ved høje værdier af l med, at hukommelsestilgange og cache-misses begynder at dominere den samlede køretid mere end selve evalueringen af den specifikke hashfunktion, idet hashtabellen bliver meget stor og mister sin hukommelseslokalitet.

Del 2

Opgave 4

Vi implementerede den 4-universelle hashfunktion $g(x)$ defineret ved tredjegradspolynomiet $g(x) = (a_0 + a_1 x + a_2 x^2 + a_3 x^3) \bmod p$, hvor p igen er Mersenne-primtallet 2 opløftet i 89 minus 1.

For at optimere denne beregning benyttede vi Horner's metode, som omskriver polynomiet algebraisk til $a_0 + x(a_1 + x(a_2 + x a_3))$. I Horner's metode trækker vi x udenfor parenteserne og reducerer antallet af dyre store multiplikationer markant i forhold til den direkte udregning af potenserne. Denne udregning ses i koden her;

```

1 y = FastModP((y * bx) + _a2);

```

```

2 y = FastModP((y * bx) + _a1);
3 y = FastModP((y * bx) + _a0);

```

For modulo-operationen udnyttede vi at p er et Mersenne-primtal. I modulær aritmetik gælder for dette p at 2 opløftet i 89 er kongruent med 1 modulo p . Ethvert tal y kan skrives som $y = q$ gange (2 opløftet i 89) + r , hvoraf y er kongruent med $q + r$ modulo p . I vores kode forinden betyder det, at vi i stedet for den dyre standard modulo operator forskyder y med 89 pladser til højre og lægger det sammen med de nederste 89 bits fundet via en bitmaske. Vi bruger vores funktion `Hash(ulong x)` til at beregne hashværdier og igen er `BigInteger` brugt til at konvertere input til et stort tal.

```

1 public BigInteger Hash(ulong x)
2 {
3     BigInteger bx = new BigInteger(x);
4     BigInteger y = _a3;
5     y = FastModP((y * bx) + _a2);
6     y = FastModP((y * bx) + _a1);
7     y = FastModP((y * bx) + _a0);
8     return y;
9 }
10
11 private BigInteger FastModP(BigInteger val)
12 {
13     BigInteger z = (val >> 89) + (val & _p);
14     if (z >= _p) z -= _p;
15     return z;
16 }

```

Opgave 5

I Count-Sketch er det essentielt at have to uafhængige hashfunktioner, $h(x)$ og $s(x)$. Vi udleder nu disse to fra vores 4-universelle funktion $g(x)$. i opgave 4. $H(x)$ bruges til at afgøre hvilken tæller der skal opdateres, og $s(x)$ afgør fortegnet.

$h(x)$ afbilder til det krævede array-indeks ved at tage de t mindst betydende bits af $g(x)$. $s(x)$ tildeler et tilfældigt fortegn (1 eller -1) til hvert element ved at aflæse den mest betydende bit (bit nummer 89) af $g(x)$. Matematisk sikrer dette at fortegnsfordelingen og indeksfordelingen arver de probabilistiske egenskaber og den 4-universelle fordeling, vi opnåede i Opgave 4. Her er koden der udleder h og s :

```

1 public (int h, int s) Evaluate(ulong x)
2 {
3     BigInteger gx = _g.Hash(x);
4     int h = (int)(gx & _mMask);
5     int msb = (int)(gx >> 88);
6     int s = 1 - 2 * msb;
7     return (h, s);
8 }

```

Opgave 6

Count-Sketch estimerer kvadratsummen $S = \sum_{x \in U} s(x)^2$ ved brug af et lille array C med $m = 2^t$ tællere i stedet for at gemme $s(x)$ præcist for hver nøgle. Dette gør algoritmen langt mere pladseffektiv end hashing med chaining, men på bekostning af præcision.

Vi anvender nu de to hashfunktioner fra opgave 5 som vi har udledt fra den 4-universelle funktion g fra opgave 4:

$$h(x) = g(x) \bmod m \in [0, m)$$
$$s(x) = 1 - 2 \left\lfloor \frac{g(x)}{2^{b-1}} \right\rfloor \in \{-1, +1\}$$

hvor $h(x)$ bestemmer hvilken tæller der opdateres, og $s(x)$ bestemmer fortegnet.

Vi initialiserer $C \leftarrow 0^m$ og behandler hvert element (x, d) i strømmen:

$$C[h(x)] \leftarrow C[h(x)] + s(x) \cdot d$$

Efter at have processeret hele strømmen beregnes estimatet:

$$X = \sum_{y \in [m]} C[y]^2$$

Fortegnet $s(x) \in \{-1, +1\}$ sikrer at kollisioner udligner hinanden i forventning. Det er vist at:

$$\mathbb{E}[X] = S \quad \text{og} \quad \text{Var}[X] \leq \frac{2S^2}{m}$$

Større m giver altså mindre varians og dermed et bedre estimat.

Vi testede med $n = 10^6$, $l = 10$ og $t = 8$ ($m = 256$ tællere) og opnåede en eksakt værdi på $S = 108,509,512$, et Count-Sketch estimat på $X = 102,504,162$ og en relativ fejl på 5.53%. Dette stemmer overens med den teoretiske standardafvigelse $\sqrt{\text{Var}[X]} \leq \frac{S\sqrt{2}}{\sqrt{m}} \approx 9,591,017$, da vores absolutte fejl på 6,005,350 ligger under denne grænse.

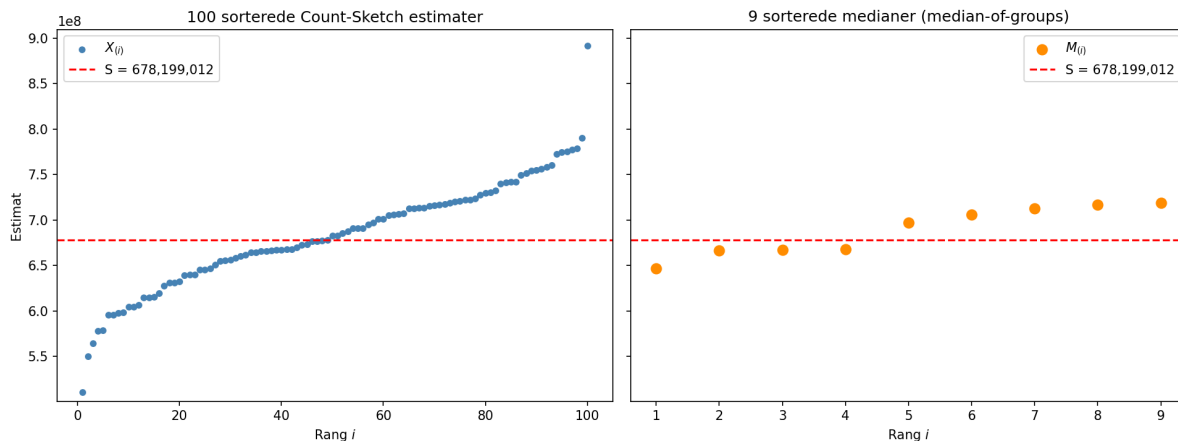
```
1 public void Update(ulong x, int d)
2 {
3     ulong h = Opgave5.H(x, _t, _a0, _a1, _a2, _a3);
4     int s = Opgave5.S(x, _a0, _a1, _a2, _a3);
5     _C[h] += s * d;
6 }
7
8 public long ComputeEstimate()
9 {
10     long X = 0;
11     foreach (long c in _C)
12         X += c * c;
13     return X;
14 }
```

Opgave 7

Vi undersøgte vi variationen i Count-Sketch ved at køre 100 uafhængige eksperimenter over en datastrøm på $n = 10^7$ elementer, hvor antallet af tællere var sat til $m = 256$

($t = 8$) og universets størrelse via $l = 14$. Da hashfunktionerne er baseret på tilfældige koefficienter, vil estimatet for kvadratsummen S variere markant pr. kørsel – i vores tilfælde fra ~ 510 mio. til ~ 891 mio. omkring den værdi $S = 678.199.012$.

Vi delte de 100 uafhængige estimater ind i 9 grupper af 11 estimater (og ignorerede den sidste værdi). Vi fandt derefter medianen af hver gruppe og sorterede de resulterende 9 medianer.



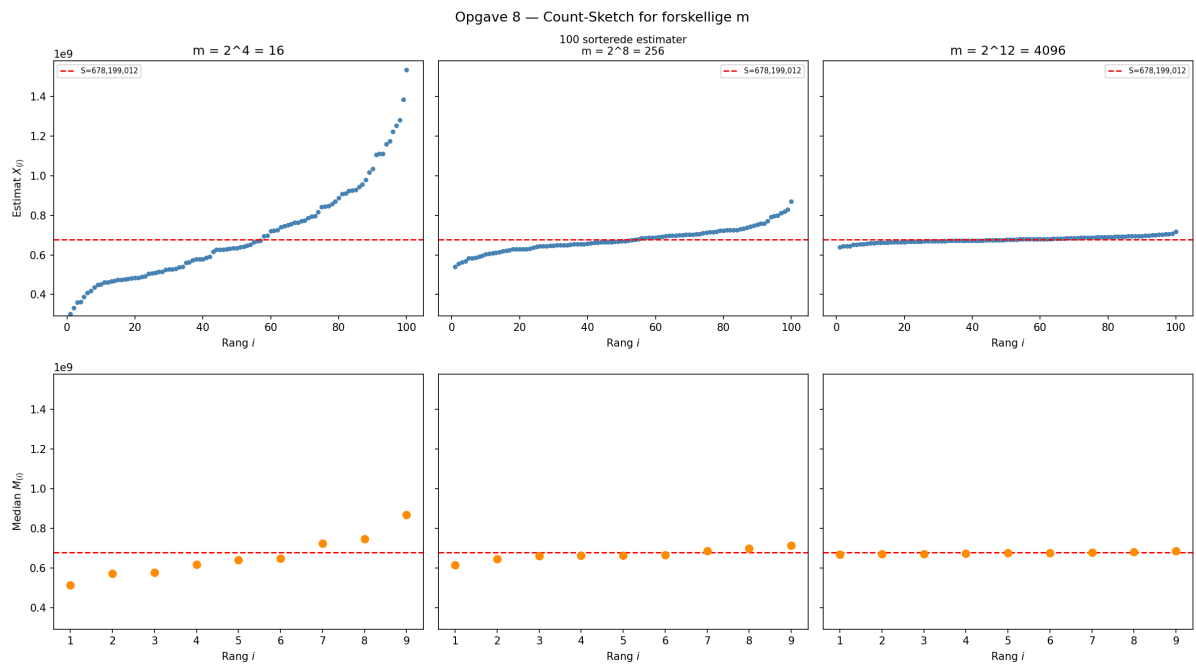
Figur 1: Spredning af 100 uafhængige Count-Sketch estimater sammenlignet med 9 gruppe-medianer for $m = 256$.

Vi ser, at hvis vi trækker medianen ud, sorterer vi de mest ekstreme outliers fra i begge retninger. De 9 medianer spændte kun fra ~ 646 mio. til ~ 719 mio.

Opgave 8

Vi kan undersøge hvilken effekt det har, at justere størrelsen af tæller-arrayet ved at afprøve $m \in \{16, 256, 4096\}$ (tilsvarende $t \in \{4, 8, 12\}$).

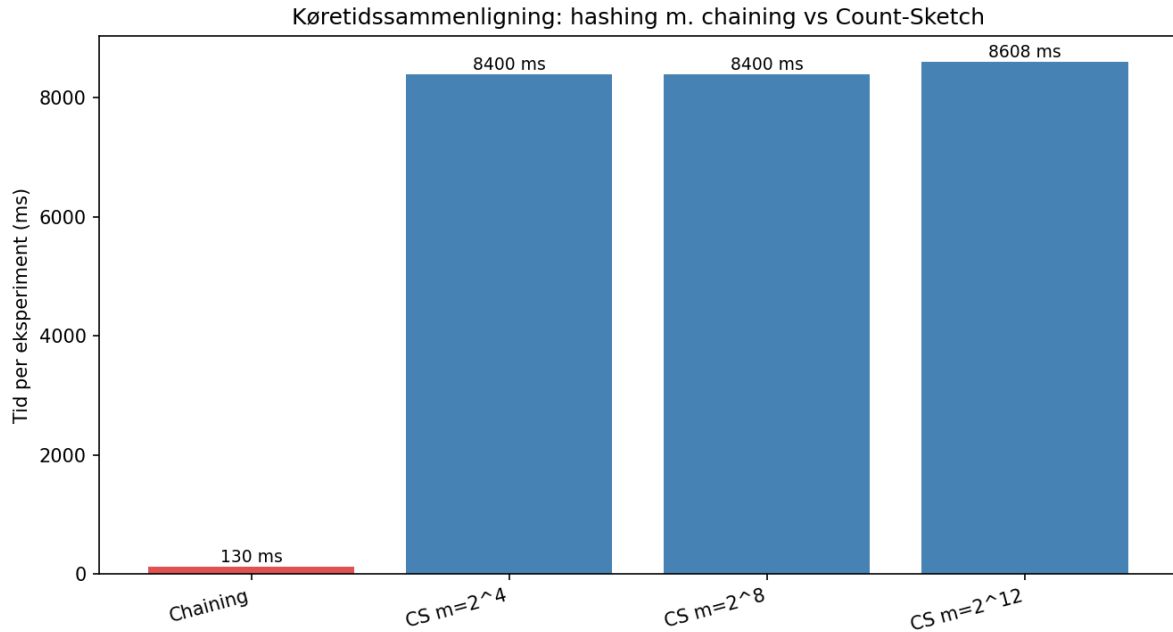
For Count-Sketch er variansen begrænset af $\text{Var}[X] \leq \frac{2S^2}{m}$. Det forventes derfor at estimaterne konvergerer tættere omkring den eksakte værdi S , når arrayets dimension vokser.



Figur 2: Udvikling i estimatets præcision og medianens stabilitet som funktion af antallet af tællere m .

Vi ser, at netop dette mønster er tydeligt. Ved $m = 16$ observeredes en voldsom spredning fra ~ 301 mio. op til over 1,5 mia. Ved $m = 256$ snævredes spredningen ind fra ~ 541 mio. til 868 mio. Ved $m = 4096$ var der en meget koncentreret spredning fra 638 mio. til 716 mio.

Udover præcision vil vi gerne vide noget om den reelle processortid. Mens Count-Sketch garanterer et lavt hukommelsesaftrek ift. en fuld hashtabel med chaining, observerede vi i praksis en markant forskel i tidsforbrug.



Figur 3: Sammenligning af køretid per eksperiment

Det tog det præcise S via hashing med chaining ~ 130 ms, mens Count-Sketch pr. eksperiment tog $\sim 8.400 - 8.600$ ms.

At chaining-metoden fremstår mere end 60 gange hurtigere fortæller ikke, at Count-Sketch metodisk er asymptotisk værre tidsmæssigt, men snarere hvorledes detaljerne i implementationen skalerer på hardware.

Hos Count-Sketch anvendes to uafhængige hashfunktioner $h(x)$ og $s(x)$, som i vores arkitektur nedarves direkte af det 3.-grads 4-universelle polynomium. At instantiere og gange BigInteger 10 millioner gange indebærer en signifikant abstraktions- og allokeringsstraf for maskinen. Det tog kørslen stort set den samme mængde tid ligegyldigt om tællerarrayet var på størrelse $m = 16$ eller $m = 4096$, hvilket viser, at overhead opstår ikke i selve Count-Sketch-opdateringen, men derimod i BigInteger-aritmetikker.

Det er dog værd at pointere, at vi har kørt chaining operationerne på en anden maskine end resten af testne. Vi mener dog, at maskinerne er tæt nok hardware-mæssig, at det ikke bør have stor indflydelse på benchmarks'ne.